

analysis.ipynb 해설지

사출성형 공정데이터 기반 품질불량 사전예측 및 검사 우선순위 결정
제6회 K-인공지능 제조데이터 분석 경진대회 · 과제 ①

구분	내용
읽는 법	노트북 셀 순서 그대로 [목적] → 코드 → 해설 → [출력 확인] (파랑) → [수정 포인트] (주황).
구성	0장 전체 구조 · A장 src 모듈 · 1~5장 노트북 셀 해설(보고서 1~4장과 대응) · 부록 용어/오류/체크리스트
주의	코드 인용은 작성 시점 기준. 코드를 고치면 해설지의 해당 셀도 함께 갱신할 것.

0. 전체 구조 한눈에 보기

프로젝트는 **로직(src/)** 과 **실행 진입점** 두 개로 나뉜다. 노트북은 계산을 새로 짜지 않고 src 함수를 불러 결과를 보여주지만 하므로, 로직을 고치면 run_all.py 와 노트북 양쪽에 동시에 반영된다.

파일	역할	언제 고치나
src/config.py	경로, 시드, CV 횟수, 검사 시나리오	실험 속도·설정 변경
src/data.py	적재, 사출·캐비티 복원, 진단, 입력 X 생성	전처리·특성 추가
src/models.py	비교 모델 정의	모델 추가·하이퍼파라미터
src/evaluate.py	그룹 CV, 임계값, 지표	지표·검증 방식 변경
run_all.py	전 과정 자동 실행 → outputs/ 저장 (재현성 10점)	거의 안 고침
analysis.ipynb	결과 시각화·해석 (보고서 그림·표 출처)	그래프·분석 추가

실행 흐름

```
data/*.csv
| load_all() — 사출(shot) · 캐비티 구조 복원
▼
진단(diagnose) —————▶ outputs/data_diagnosis.csv
| holdout_split() — 사출 단위 80/20 분리
|— dev(80%) — grouped_cv(모델3 × 특성셋2) —▶ outputs/cv_folds.csv, cv_summary.csv, oof_dev.csv
|               ↳ PR-AUC 최고 조합 = 최종모델
|— test(20%) — 최종모델 예측 —————▶ outputs/test_predictions.csv, test_metrics.json

analysis.ipynb : 위 결과 파일 + src 함수로 그림/표 생성
```

핵심 설계 3가지 (보고서에서 반복해서 강조할 것)

- **2캐비티 구조 → 사출 단위 그룹 CV**: 같은 사출의 두 제품(공정값 동일)이 학습/평가에 나뉘면 성능이 부풀려진다.
- **임계값은 학습 fold 내부에서만 선정**: 평가 데이터로 임계값을 고르면 F1 이 과대평가된다.
- **PR-AUC 주지표 + Recall@k%**: 불량 1~2% 의 극단적 불균형 → 정확도는 무의미, 검사 우선순위로 바로 연결되는 지표 사용.

A. src 모듈 해설

노트북 셀을 이해하려면 먼저 이 네 파일이 무엇을 하는지 알아야 한다. 전체 코드는 각 파일을 직접 열어 확인한다.

src/config.py — 경로·상수

- 모든 설정값이 한 곳에 있다. CV 반복 횟수(N_REPEATS), fold 수(N_SPLITS), 테스트 비율(TEST_FRAC), 검사 시나리오(TOP_K).
- 수정: 빠르게 실험할 때 N_REPEATS 를 5 → 1 로 줄이면 실행 시간이 약 1/5.

src/data.py — 데이터 적재·구조 복원·진단

- **add_shot_structure**: 직전 행과 공정값 차이 합이 0 이면 같은 사출로 본다 → shot_id, cavity 생성. 이 프로젝트의 핵심 전처리.
- **diagnose**: 1장 진단표. **scaling_check**: 라벨/비라벨 표준화 확인.
- **feature_matrix**: 모델 입력 X 생성. 상수 컬럼 제거 + is_rg3(제품 구분) + (선택) cavity.
- 수정: 새 특성(예: 변수 간 비율·차이)을 추가하려면 feature_matrix 안에서 X['새이름'] = ... 로 넣는다.

src/models.py — 비교 모델

- LR_baseline(표준화+로지스틱, 불균형 가중), RandomForest, LightGBM. 모두 불량 클래스에 가중치를 준다.
- 수정: 모델 추가는 get_models() 딕셔너리에 '이름': 모델 한 줄 추가 → run_all 과 노트북에 자동 반영.

src/evaluate.py — 평가 로직

- **grouped_cv**: StratifiedGroupKFold(사출 단위 그룹 + 불량 비율 유지) 5-fold × 5회 반복.
- **best_threshold / inner_oof**: 학습 fold 내부 예측으로만 F1 최대 임계값 선정.
- **recall_at_k**: 상위 k% 검사 시 포착률. **metrics**: 지표 묶음 계산.
- 수정: 지표 추가는 metrics() 딕셔너리에 한 줄 추가.

핵심 코드: 사출 구조 복원 (src/data.py)

```
cols = process_cols(df)                                # 공정변수 24개
new_shot = df[cols].diff().abs().sum(axis=1).ne(0)      # 직전 행과 하나라도 다르면 새 사출
df['shot_id'] = new_shot.cumsum() - 1                  # 새 사출마다 번호 +1
df['cavity'] = df.groupby('shot_id').cumcount()         # 사출 안에서 0, 1 순번
```

가정: 행 순서 = 캐비티 순서. 데이터에 캐비티 번호가 명시되지 않았으므로 보고서에 가정으로 밝힌다.

핵심 코드: 누수 없는 평가 (src/evaluate.py)

```
cv = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=SEED + r)
for tr, va in cv.split(X, y, groups):                  # groups = 제품_사출번호
    m = clone(model).fit(X.iloc[tr], y[tr])
    thr = best_threshold(y[tr], inner_oof(model, X.iloc[tr], y[tr], groups[tr], ...)) # 학습 fold 안에서만
    p = m.predict_proba(X.iloc[va])[:, 1]              # 평가 fold 는 예측에만 사용
```

0장. 환경 설정

[셀 1] 0-1. 라이브러리와 설정 불러오기

목적 노트북 전체에서 쓰는 도구와 우리 프로젝트 함수(src/)를 한 번에 불러온다.

```
import json
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.base import clone

from src.config import INDEX_COL, OUT_DIR, SEED, TARGET
from src.data import diagnose, feature_matrix, load_all, load_labeled, process_cols, scaling_check
from src.evaluate import best_threshold, inner_oof, metrics
from src.models import get_models
from run_all import holdout_split

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 50)
plt.rcParams["font.family"] = "Malgun Gothic" # 한글 폰트 (Windows)
plt.rcParams["axes.unicode_minus"] = False
sns.set_style("whitegrid")
```

- **src.config**: 경로(OUT_DIR), 정답 컬럼명(TARGET), 시드(SEED) 같은 상수.
- **src.data**: 데이터 적재·사출 구조 복원·진단 함수. **src.evaluate**: 평가 지표 함수. **src.models**: 비교 모델 3종.
- **from run_all import holdout_split**: 테스트셋 분리 방식을 run_all.py 와 똑같이 쓰기 위해 가져온다 (노트북과 스크립트 결과가 어긋나지 않게).
- Malgun Gothic 설정은 그래프 한글 깨짐 방지용이다.

[출력 확인]

- 에러 없이 끝나면 OK. **ModuleNotFoundError: src** 가 나면 노트북을 injection_molding 폴더에서 연 것이 아니다.

[수정 포인트]

- Mac 에서 실행한다면 font.family 를 'AppleGothic' 으로 바꾼다.

[셀 2] 0-2. 파이프라인 결과가 없으면 먼저 실행

목적 2장에서 읽을 CV 결과 파일(outputs/cv_folds.csv)이 없으면 run_all.main() 으로 전체 파이프라인을 돌린다.

```
# CV 결과가 없으면 전체 파이프라인을 먼저 실행 (CPU 약 10분)
if not (OUT_DIR / "cv_folds.csv").exists():
    import run_all
    run_all.main()
```

- CV(교차검증)는 모델 3종 × 특성셋 2종 × 25회 학습이라 CPU 로 약 10분 걸린다. 노트북에서 매번 돌리지 않도록 결과를 파일로 저장해 두고 재사용하는 구조다.
- 이미 파일이 있으면 아무 일도 하지 않는다.

[출력 확인]

- 처음 실행 시 'done LR_baseline ...' 같은 진행 로그가 6줄 나온다.

[수정 포인트]

- 모델·CV 설정을 바꿨다면 **outputs/cv_folds.csv** 를 지우고 이 셀을 다시 실행해야 새 결과가 반영된다.

1장. 데이터 이해 및 진단 (보고서 1장 · 15점)

[셀 3] 1-0. 데이터 적재

목적 CN7-RG3 라벨 데이터를 읽어 한 표로 합치고, 사출(shot) 구조 컬럼을 붙인다.

```
df = load_all()
FEATS = process_cols(load_labeled("cn7"))
print("행 수:", len(df), "/ 공정변수:", len(FEATS))
df.head()
```

- load_all() 이 추가하는 컬럼: **shot_id**(같은 사출 번호), **cavity**(사출 내 순서 0/1), **shot_size**(사출당 제품 수), **product**(cn7/rg3), **group**(제품+사출번호, CV 그룹용).
- FEATS = 원본 공정변수 24개 목록. 이후 분석 대부분이 이 목록을 기준으로 한다.

[출력 확인]

- 행 수 2393, 공정변수 24개가 찍히면 정상.

[셀 4] 1-1. 데이터 진단표

목적 보고서 1장에 그대로 옮길 제품별 요약표를 만든다.

```
diag = diagnose(df)
diag.T
```

- fail_rate: 불량률. constant_cols: 값이 하나뿐이라 쓸모없는 변수(Clamp_Open_Position).
- shots / shots_2cavity: 사출 수와 그중 제품 2개가 나온 사출 수 → 2캐비티 구조의 근거.
- fail_at_cavity0: 첫 번째 캐비티에서 나온 불량 수 → 캐비티 위치가 중요하다는 근거.
- conflicting_patterns / fails_in_conflicting: 공정값이 완전히 같은데 양품·불량이 섞인 패턴 수와 그 안의 불량 수.

[출력 확인]

- CN7 불량 17 (1.40%), RG3 불량 25 (2.12%).
- 첫 캐비티 불량: CN7 13/17, RG3 23/25.
- 충돌 패턴 내 불량: CN7 11/17, RG3 25/25.

[수정 포인트]

- 보고서용으로 컬럼명을 한글로 바꾸려면 diag.rename(columns={...}) 를 덧붙인다.

[셀 5] 1-2. 클래스 불균형 그래프

목적 양품/불량 개수와 불량률을 막대그래프로 보여준다.

```
fig, ax = plt.subplots(1, 2, figsize=(10, 3.5))
cnt = df.groupby(["product", TARGET]).size().unstack()
cnt.plot.bar(ax=ax[0], rot=0, color=["#9bb7d4", "#d9534f"])
ax[0].set_title("제품별 양품(0) / 불량(1) 수")
(df.groupby("product")[TARGET].mean() * 100).plot.bar(ax=ax[1], rot=0, color="#d9534f")
ax[1].set_title("제품별 불량률 (%)")
plt.tight_layout(); plt.show()
```

- 왼쪽: 제품별 양품(0)·불량(1) 수. 오른쪽: 불량률(%).
- 불량률이 1~2% 뿐이라 '전부 양품'이라고 찍어도 정확도 98% 가 나온다 → 정확도(Accuracy) 대신 PR-AUC-F1 을 써야 하는 이유.

[출력 확인]

- 불량 막대가 거의 보이지 않을 정도로 작아야 정상.

[수정 포인트]

- 색상은 color=[양품색, 불량색] 리스트에서 바꾼다.

[셀 6] 1-3. 2캐비티 사출 구조 표

목적 사출 단위 통계와 캐비티(0/1)별 불량률을 표로 확인한다.

```
shot_tbl = (df.groupby("product")
            .apply(lambda d: pd.Series({
                "사출 수": d.shot_id.nunique(),
                "2개 제품 사출": (d.groupby("shot_id").size() == 2).sum(),
                "1개 제품 사출": (d.groupby("shot_id").size() == 1).sum(),
                "불량 사출": (d.groupby("shot_id")[TARGET].sum() > 0).sum(),
            })))
display(shot_tbl)

cav = pd.crosstab([df["product"], df["cavity"]], df[TARGET], margins=False)
cav["불량률(%)"] = (cav[1] / cav.sum(axis=1) * 100).round(2)
display(cav)
```

- shot_tbl: 제품별 사출 수, 2개/1개 제품 사출 수, 불량률 하나라도 있는 사출 수.
- pd.crosstab: (제품, 캐비티) × (양품, 불량) 교차표. 불량률(%) 컬럼을 추가로 계산한다.
- 왜 중요한가: 한 사출의 두 제품은 공정값이 완전히 같다. 둘이 학습/검증에 나뉘면 모델이 '정답을 본' 셈이 되어 성능이 부풀려진다 → 사출 단위 그룹 CV 를 쓰는 근거.

[출력 확인]

- 캐비티 0 의 불량률이 캐비티 1 보다 확연히 높아야 한다.

[셀 7] 1-4. 캐비티별 불량률 그래프

목적 1-3 의 캐비티별 불량률을 막대그래프로 시각화 (보고서 삽입용).

```
fig, ax = plt.subplots(figsize=(6, 3.5))
(df.groupby(["product", "cavity"])[TARGET].mean() * 100).unstack().plot.bar(ax=ax, rot=0)
ax.set_title("캐비티 위치별 불량률 (%)"); ax.set_ylabel("%")
plt.tight_layout(); plt.show()
```

[출력 확인]

- cavity 0 막대가 높게 나온다.

[수정 포인트]

- 보고서용으로 저장하려면 plt.show() 앞에 plt.savefig('outputs/fig_cavity.png', dpi=200) 추가.

[셀 8] 1-5. 라벨 충돌 확인

목적 공정값 패턴이 같은 행끼리 묶어 양품·불량이 섞인 경우를 센다.

```
rows = []
for p, d in df.groupby("product"):
    pat = d.groupby(FEATS)[TARGET].agg(["size", "nunique", "sum"])
    rows.append({"product": p,
                "불량 수": int(d[TARGET].sum()),
                "충돌 패턴 내 불량": int(pat.loc[pat["nunique"] > 1, "sum"].sum()),
                "순수 불량 패턴": int((pat["sum"] == pat["size"]).sum())})
pd.DataFrame(rows)
```

- groupby(FEATS): 24개 공정변수 값이 모두 같은 행끼리 묶는다.
- nunique > 1 이면 같은 공정값에 양품과 불량이 모두 있다는 뜻 → 공정변수만으로는 원리적으로 구분 불가.
- 보고서 메시지: 'RG3 불량률 전부 같은 공정값의 양품 짝이 존재 → 공정 데이터만으로는 한계, 캐비티(금형) 정보가 필요'.

[출력 확인]

- RG3: 불량 25, 충돌 패턴 내 불량 25.

[셀 9] 1-6. 결측·상수·스케일 점검

목적 결측치, 상수 컬럼, 라벨/비라벨 데이터의 표준화 상태를 확인한다.

```
print("결측치 합계:", int(df[FEATS].isna().sum().sum()))
print("상수 컬럼:", [c for c in FEATS if df[c].nunique() == 1])
scaling_check() # 라벨/비라벨 데이터가 각자 따로 표준화됨 → 결합하지 않음
```

- scaling_check(): 각 파일의 평균·표준편차를 보여준다. 라벨 데이터도 비라벨 데이터도 각자 평균 0·표준편차 1 → 각 파일을 따로 표준화했다는 증거.
- 따로 표준화된 데이터는 같은 값이 같은 실제 값을 뜻하지 않으므로, 비라벨 7만행을 학습에 그대로 섞지 않았다.

[출력 확인]

- 결측 0, 상수 컬럼 ['Clamp_Open_Position'], max_abs_mean 0.

[셀 10] 1-7. 생산 순번별 불량 분포

목적 원본 순번(Unnamed: 0)을 x축으로 불량 발생 위치를 찍는다.

```
fig, axes = plt.subplots(2, 1, figsize=(12, 4), sharey=True)
for ax, (p, d) in zip(axes, df.groupby("product")):
    ax.scatter(d[INDEX_COL], d[TARGET], s=4, alpha=.6, c=d[TARGET], cmap="coolwarm")
    ax.set_title(f"{p.upper()} - 생산 순번별 불량 발생")
    ax.set_yticks([0, 1])
plt.tight_layout(); plt.show()
```

- CN7 불량은 순번 47~120 구간에 몰려 있다 → 특정 시기에 일어난 '사건'일 가능성.
- RG3 불량은 전 구간에 흩어져 있다.

[출력 확인]

- 위쪽 줄(1)에 찍힌 점의 위치를 본다.

[수정 포인트]

- 산점도가 너무 뾰뾰하면 s(점 크기)와 alpha(투명도)를 조정.

[셀 11] 1-8. 양품 vs 불량 평균 차이 히트맵

목적 변수별로 '불량 평균 - 양품 평균'을 계산해 어떤 공정변수가 불량과 관련 있는지 한눈에 본다.

```
use = [c for c in FEATS if df[c].nunique() > 1]
diff = (df.groupby(["product", TARGET])[use].mean()
        .groupby(level=0).diff().dropna().droplevel(1))
fig, ax = plt.subplots(figsize=(12, 3))
sns.heatmap(diff, cmap="RdBu_r", center=0, annot=True, fmt=".1f", ax=ax, cbar=False)
ax.set_title("불량 평균 - 양품 평균 (표준화 단위)")
plt.tight_layout(); plt.show()
```

- use = 상수 컬럼을 뺀 변수 목록.
- groupby([product, TARGET]).mean() 후 diff() → 불량 - 양품. 값이 표준화 단위라 ±1 이면 표준편차 1개만큼 차이.
- 빨강: 불량일 때 더 큼, 파랑: 불량일 때 더 작음.

[출력 확인]

- 절댓값이 큰 칸이 후보 영향변수.

[셀 12] 1-9. 상위 차이 변수 박스플롯

목적 1-8 에서 차이가 큰 변수 6개를 골라 양품/불량 분포를 비교한다.

```
top = diff.abs().max().sort_values(ascending=False).index[:6]
fig, axes = plt.subplots(2, 3, figsize=(13, 6))
for ax, c in zip(axes.ravel(), top):
    sns.boxplot(data=df, x="product", y=c, hue=TARGET, ax=ax, fliersize=2)
    ax.set_title(c); ax.legend_.remove()
plt.tight_layout(); plt.show()
```

- top = 차이 절댓값 기준 상위 6개. 개수를 바꾸려면 [:6] 과 subplots(2, 3) 을 함께 바꾼다.

[출력 확인]

- 불량(주황) 박스가 양품과 확연히 다른 위치에 있는 변수를 메모.

[수정 포인트]

- [:6] → [:9], subplots(2, 3) → subplots(3, 3) 처럼 짝을 맞춘다.

[셀 13] 1-10. 공정변수 상관 히트맵

목적 변수 간 상관을 제품별로 보여준다. 거의 같은 정보를 담은 변수 묶음을 찾는 용도.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
for ax, (p, d) in zip(axes, df.groupby("product")):
    sns.heatmap(d[use].corr(), cmap="RdBu_r", center=0, ax=ax, cbar=False)
    ax.set_title(f"{p.upper()} 공정변수 상관")
plt.tight_layout(); plt.show()
```

- 예: Barrel_Temperature_1~6 끼리, 시간 관련 변수끼리 상관이 높게 나오면 SHAP 해석 시 '묶음'으로 해석해야 한다.

[수정 포인트]

- 숫자를 보고 싶으면 annot=True, fmt='.1f' 추가 (칸이 많아 글씨가 작아짐).

2장. AI 예측모델 개발 및 성능평가 (보고서 2장 · 40점)

[셀 14] 2-1. CV 결과 요약표

목적 run_all 이 저장한 fold 별 성능(cv_folds.csv)을 읽어 모델·특성셋별 평균±표준편차를 만든다.

```
cv = pd.read_csv(OUT_DIR / "cv_folds.csv")
MET = ["PR_AUC", "ROC_AUC", "F1", "Precision", "Recall", "Recall@5%", "Recall@10%", "Recall@20%"]
summary = cv.groupby(["scope", "features", "model"])[MET].agg(["mean", "std"]).round(3)
summary.loc["all"]
```

- scope: all(두 제품 합쳐서), cn7, rg3.
- features: process_only(공정변수만) / process+cavity(캐비티 추가).
- PR_AUC: 불량을 위로 얼마나 잘 줄세우는지(임계값 무관, 주지표). F1: 임계값 적용 후 정밀도·재현율 조화평균.
Recall@k%: 상위 k% 만 검사할 때 잡는 불량 비율.

[출력 확인]

- 현재 결과: RandomForest + 캐비티가 PR-AUC 0.114 로 최고 (LR 베이스라인 공정변수만 0.088).

[수정 포인트]

- 표 전체를 보려면 summary (loc 없이) 를 출력. 제품별은 summary.loc['cn7'].

[셀 15] 2-2. PR-AUC 분포 박스플롯

목적 25개 fold 의 PR-AUC 흠어짐을 모델·특성셋·제품별로 비교한다.

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharey=False)
for ax, sc in zip(axes, ["all", "cn7", "rg3"]):
    sns.boxplot(data=cv[cv.scope == sc], x="model", y="PR_AUC", hue="features", ax=ax)
    ax.set_title(f"PR-AUC 분포 ({sc})"); ax.set_xlabel("")
plt.tight_layout(); plt.show()
```

- 평균만 보면 우연한 차이인지 알 수 없다. 박스가 겹치면 차이가 불확실하다는 뜻 → 보고서에 '분산이 크다'는 점을 솔직히 적는 근거.

[출력 확인]

- RG3 패널에서 process_only 박스가 바닥 근처인지 확인.

[셀 16] 2-3. 평균 성능 막대 + 무작위 기준선

목적 PR-AUC, F1, Recall@20% 를 막대로 그리고, 무작위 모델의 PR-AUC(= 불량률 0.018)를 점선으로 표시.

```
base = df[TARGET].mean()
tbl = cv[cv.scope == "all"].groupby(["model", "features"])[["PR_AUC", "F1", "Recall@20%"]].mean().unstack()
tbl.plot.bar(figsize=(12, 4), rot=0)
plt.axhline(base, ls="--", c="gray", label=f"무작위 PR-AUC ≈ {base:.3f}")
plt.title("모델 × 특성셋 평균 성능 (전체)"); plt.legend(bbox_to_anchor=(1, 1)); plt.tight_layout(); plt.show()
```

- PR-AUC 는 무작위일 때 불량률과 같아진다. 점선보다 몇 배 높은지가 모델의 실제 가치.

[셀 17] 2-4. 최종모델 선정 및 테스트 성능

목적 CV PR-AUC 최고 조합을 고르고, 개발셋 전체로 다시 학습해 테스트셋에서 평가한다.

```
allscope = cv[cv.scope == "all"].groupby(["model", "features"])["PR_AUC"].mean()
BEST_MODEL, BEST_FS = allscope.idxmax()
USE_CAV = BEST_FS == "process+cavity"
print(f"최종모델: {BEST_MODEL} / {BEST_FS} (CV PR-AUC {allscope.max():.3f})")

dev, test = holdout_split(df)
X_dev, X_test = feature_matrix(dev, USE_CAV), feature_matrix(test, USE_CAV)
y_dev, y_test = dev[TARGET].to_numpy(), test[TARGET].to_numpy()

model = get_models()[BEST_MODEL]
THR = best_threshold(y_dev, inner_oof(model, X_dev, y_dev, dev["group"].to_numpy(), SEED))
final = clone(model).fit(X_dev, y_dev)
p_test = final.predict_proba(X_test)[: , 1]

res = {"all": metrics(y_test, p_test, THR)}
for p in ["cn7", "rg3"]:
    m = test["product"].to_numpy() == p
    res[p] = metrics(y_test[m], p_test[m], THR)
print("임계값:", round(THR, 4))
pd.DataFrame(res).round(3)
```

- holdout_split: run_all.py 와 동일한 방식(사출 단위·불량 총화, 20%)으로 테스트셋을 나눈다.
- THR(임계값): 개발셋 내부 3-fold 예측으로만 F1 최대 지점을 고른다. 테스트셋은 임계값 선정에 쓰지 않는다 (정보 누수 방지).
- final: 개발셋 전체로 학습한 최종모델. p_test: 테스트 불량확률.

[출력 확인]

- 테스트 불량이 8건뿐이라 수치 변동이 크다. 보고서에서는 CV 결과를 주 근거로, 테스트는 '보조 확인'으로 쓴다.

[수정 포인트]

- 특정 모델을 강제로 쓰려면 BEST_MODEL, BEST_FS = 'LightGBM', 'process+cavity' 처럼 직접 지정.

[셀 18] 2-5. 혼동행렬 · PR 곡선

목적 테스트셋에서 맞힌/틀린 개수와 제품별 PR 곡선을 그린다.

```
from sklearn.metrics import ConfusionMatrixDisplay, PrecisionRecallDisplay

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ConfusionMatrixDisplay.from_predictions(y_test, (p_test >= THR).astype(int), ax=ax[0],
                                       display_labels=["양품", "불량"], colorbar=False)
ax[0].set_title("테스트 혼동행렬")
for p in ["cn7", "rg3"]:
    m = test["product"].to_numpy() == p
    PrecisionRecallDisplay.from_predictions(y_test[m], p_test[m], ax=ax[1], name=p.upper())
ax[1].set_title("테스트 PR 곡선")
plt.tight_layout(); plt.show()
```

- 혼동행렬: 왼쪽 위 = 양품을 양품으로(TN), 오른쪽 아래 = 불량을 불량으로(TP), 오른쪽 위 = 오경보(FP), 왼쪽 아래 = 미탐지(FN).
- PR 곡선: 오른쪽 위에 가까울수록 좋다. 곡선 아래 면적 = PR-AUC(AP).

[셀 19] 2-6. 확률 보정 곡선

목적 모델이 말하는 '불량확률 30%' 가 실제로 30% 인지 점검한다.

```
from sklearn.calibration import calibration_curve

oof = pd.read_csv(OUT_DIR / "oof_dev.csv")
frac, mean_p = calibration_curve(oof[TARGET], oof["oof_prob"], n_bins=8, strategy="quantile")
plt.figure(figsize=(4.5, 4))
plt.plot(mean_p, frac, "o-", label=BEST_MODEL)
plt.plot([0, mean_p.max()], [0, mean_p.max()], "--", c="gray", label="완전 보정")
plt.xlabel("예측 불량확률"); plt.ylabel("실제 불량비율"); plt.title("보정 곡선 (dev OOF)")
plt.legend(); plt.tight_layout(); plt.show()
```

- 점이 점선(완전 보정) 위에 있으면 확률을 그대로 믿을 수 있다. 벗어나면 확률 보정(Platt/Isotonic)이 필요 → 창의성 항목의 '확률보정' 재료.
- quantile 구간 8개로 나눠 각 구간의 평균 예측확률(x)과 실제 불량비율(y)을 비교.

[수정 포인트]

- 보정을 적용하려면 `sklearn.calibration.CalibratedClassifierCV(model, method='sigmoid', cv=3)` 로 감싸 비교해 본다.

3장. 영향요인 및 오류분석 (보고서 3장 · 15점)

[셀 20] 3-1. SHAP 요약 그래프

목적 최종모델이 불량확률을 올리거나 내릴 때 어떤 변수를 쓰는지 보여준다.

```
import shap

explainer = shap.TreeExplainer(final)
sv = explainer.shap_values(X_dev)
sv1 = sv[0, 1] if np.ndim(sv) == 3 else (sv[1] if isinstance(sv, list) else sv) # 불량 클래스
shap.summary_plot(sv1, X_dev, max_display=15, show=False)
plt.title("SHAP - 불량확률 기여"); plt.tight_layout(); plt.show()
```

- 점 하나 = 제품 하나. x축 = 그 변수가 불량확률을 얼마나 밀어올렸나(+)/내렸나(). 색 = 변수 값(빨강 높음, 파랑 낮음).
- sv1: RandomForest 는 클래스별 SHAP 이 나오므로 불량(1) 클래스만 꺼낸다. 라이브러리 버전에 따라 모양이 달라 np.ndim 으로 분기.

[출력 확인]

- cavity 와 is_rg3 가 상위에 나오는지 확인.

[수정 포인트]

- RandomForest 500그러라 수 분 걸릴 수 있다. 빠르게 보려면 X_dev.sample(500, random_state=0) 로 줄인다.

[셀 21] 3-2. 주요 변수 · 의존도 그래프

목적 SHAP 평균 절댓값으로 중요도 순위를 매기고, 상위 공정변수 2개(TOP2)의 의존도 그래프를 그린다.

```
imp = pd.Series(np.abs(sv1).mean(0), index=X_dev.columns).sort_values(ascending=False)
TOP2 = [c for c in imp.index if c not in ("is_rg3", "cavity")][:2]
print("상위 변수:", imp.head(10).round(4).to_dict())
for c in TOP2:
    shap.dependence_plot(c, sv1, X_dev, interaction_index="auto", show=False)
plt.tight_layout(); plt.show()
```

- TOP2 는 cavity-is_rg3 를 제외한 공정변수 중 상위 2개. 3-3 상호작용 분석에 쓰인다.
- dependence_plot: x = 변수 값, y = SHAP. interaction_index='auto' 가 가장 강하게 얹힌 다른 변수로 색을 칠한다 → 상호작용 단서.

[출력 확인]

- 출력된 '상위 변수' 딕셔너리를 보고서 표로 옮긴다.

[수정 포인트]

- 다른 변수를 보고 싶으면 TOP2 = ['변수A', '변수B'] 로 직접 지정.

[셀 22] 3-3. 상위 2개 변수 구간별 불량률

목적 TOP2 두 변수를 각각 4구간(사분위)으로 나눠 조합별 실제 불량률 히트맵을 만든다.

```
tmp = dev.copy()
for c in TOP2:
    tmp[c + "_bin"] = pd.qcut(tmp[c], 4, duplicates="drop")
pv = tmp.pivot_table(index=TOP2[0] + "_bin", columns=TOP2[1] + "_bin", values=TARGET, aggfunc="mean") * 100
plt.figure(figsize=(7, 4.5))
sns.heatmap(pv, annot=True, fmt=".1f", cmap="Reds")
plt.title(f"{TOP2[0]} × {TOP2[1]} 구간별 불량률 (%)"); plt.tight_layout(); plt.show()
```

- 특정 칸만 불량률이 높으면 '두 조건이 동시에 만족될 때 불량'이라는 상호작용 근거가 된다.
- qcut(..., duplicates='drop'): 같은 값이 많아 구간이 겹치면 자동으로 합친다 (그래서 4구간보다 적을 수 있음).

[수정 포인트]

- 구간 수는 qcut 의 4 를 3~5 로 조정. 불량이 적어 구간을 너무 잘게 나누면 칸마다 0/1건이 되어 의미가 없다.

[셀 23] 3-4. 판정 유형 집계 (TP/FN/FP/TN)

목적 개발셋 OOF 예측(학습에 안 쓰인 fold 에서 나온 예측)으로 각 행을 4가지 유형으로 분류한다.

```
oof["error"] = oof["error"].fillna("OK")
oof["type"] = np.select(
    [(oof[TARGET] == 1) & (oof.error == "OK"), oof.error == "FN", oof.error == "FP"],
    ["TP", "FN", "FP"], "TN")
display(pd.crosstab([oof["product"], oof["cavity"]], oof["type"]))
```

- TP 불량→불량(적중), FN 불량→양품(미탐지), FP 양품→불량(오경보), TN 양품→양품.
- OOF 를 쓰는 이유: 학습에 쓴 데이터로 오류를 분석하면 오류가 거의 안 보인다.

[출력 확인]

- (제품, 캐비티)별로 FN-FP 가 어디에 몰렸는지 본다.

[셀 24] 3-5. 유형별 공정변수 평균 히트맵

목적 TP/FN/FP/TN 그룹마다 공정변수 평균을 비교한다.

```
prof = oof.groupby("type")[use].mean().loc[["TP", "FN", "FP", "TN"]]
plt.figure(figsize=(13, 3.2))
sns.heatmap(prof, cmap="RdBu_r", center=0, annot=True, fmt=".1f", cbar=False)
plt.title("판정 유형별 공정변수 평균 (표준화 단위)"); plt.tight_layout(); plt.show()
```

- FN 이 TP 와 비슷하면 → 모델이 구분할 신호는 있는데 임계값이 높다. FN 이 TN 과 비슷하면 → 공정값상 양품과 구분 불가(라벨 충돌).
- FP 가 TP 와 비슷하면 → '불량처럼 보이는 양품' 조건. 보고서 3장 FP 분석의 핵심.

[셀 25] 3-6. 변수 구간별 FN-FP 비율

목적 중요 변수 5개 각각을 4구간으로 나눠 구간별 미탐지율·오경보율을 계산한다.

```
# 오류가 집중되는 공정조건: 변수 구간별 FN / FP 비율
def error_rate_by_bin(col, q=4):
    b = pd.qcut(oof[col], q, duplicates="drop")
    g = oof.groupby(b)
    return pd.DataFrame({
        "n": g.size(),
        "불량 수": g[TARGET].sum(),
        "FN 비율(불량 중)": g.apply(lambda d: (d.type == "FN").sum() / max(1, d[TARGET].sum())),
        "FP 비율(양품 중)": g.apply(lambda d: (d.type == "FP").sum() / max(1, (d[TARGET] == 0).sum())),
    }).round(3)

for c in imp.index.drop(["is_rg3", "cavity"], errors="ignore")[:5]:
    print(f"■ {c}")
    display(error_rate_by_bin(c))
```

- FN 비율(불량 중) = 해당 구간 불량 중 놓친 비율. FP 비율(양품 중) = 해당 구간 양품 중 잘못 경보한 비율.
- 특정 구간에 비율이 튀면 '이 조건에서 모델이 실패한다' 로 보고서에 적는다.

[출력 확인]

- 불량 수가 0~1 인 구간의 FN 비율은 의미가 약하니 '불량 수' 컬럼과 함께 본다.

[수정 포인트]

- 분석 변수 개수는 [:5], 구간 수는 error_rate_by_bin(c, q=4) 의 q 로 조정.

[셀 26] 3-7. 생산 순번별 OOF 불량확률

목적 시간 순서대로 예측확률을 찍고 TP/FN/FP/TN 을 색으로 구분한다. 점선 = 임계값.

```
# 생산 순번에 따른 예측확률 - 오류가 특정 시기에 몰리는지
fig, axes = plt.subplots(2, 1, figsize=(12, 5))
for ax, (p, d) in zip(axes, oof.groupby("product")):
    for t, col in [("TN", "#cccccc"), ("FP", "orange"), ("TP", "green"), ("FN", "red")]:
        s = d[d.type == t]
        ax.scatter(s[INDEX_COL], s["oof_prob"], s=10 if t == "TN" else 30, c=col, label=t)
    ax.axhline(THR, ls="--", c="k", lw=.8)
    ax.set_title(f"{p.upper()} - 생산 순번별 OOF 불량확률"); ax.legend(ncol=4, fontsize=8)
plt.tight_layout(); plt.show()
```

- CN7 에서 FN 이 특정 시기에 몰리면 '시기성 사건(설비 이상 등)을 공정변수가 다 설명하지 못함' 으로 해석.
- FP 가 불량 구간 주변에 몰리면 '공정 이상 구간은 잡지만 양품까지 함께 경보' 로 해석 → 현장에서는 구간 단위 점검으로 연결 가능.

4장. 현장 활용방안 (보고서 4장 · 10점)

[셀 27] 4-1. 검사 우선순위 이득곡선

목적 불량확률이 높은 순으로 검사할 때, 검사 비율(x) 대비 누적 불량 포착률(y)을 그린다.

```
def gain_curve(y, p):
    order = np.argsort(-p)
    return np.arange(1, len(y) + 1) / len(y), np.cumsum(y[order]) / y.sum()

plt.figure(figsize=(6, 4.5))
for p, d in oof.groupby("product"):
    x, r = gain_curve(d[TARGET].to_numpy(), d["oof_prob"].to_numpy())
    plt.plot(x * 100, r * 100, label=p.upper())
plt.plot([0, 100], [0, 100], "--", c="gray", label="무작위 검사")
for k in (5, 10, 20):
    plt.axvline(k, c="lightgray", lw=.8)
plt.xlabel("검사 비율 (%)"); plt.ylabel("불량 포착률 (%)"); plt.title("검사 우선순위 이득곡선 (dev 00F)")
plt.legend(); plt.tight_layout(); plt.show()
```

- 점선(대각선) = 무작위로 검사할 때. 곡선이 왼쪽 위로 치솟을수록 적게 검사하고 많이 잡는다.
- 세로선 5/10/20% 는 현장 검사 여력 시나리오.

[출력 확인]

- CN7 곡선이 RG3 보다 훨씬 위에 있어야 정상.

[셀 28] 4-2. 검사 비율별 포착률 표

목적 검사 비율 5/10/20/30% 에서 몇 개를 검사해 불량 몇 개를 잡는지 표로 정리한다.

```
rows = []
for p, d in oof.groupby("product"):
    y, s = d[TARGET].to_numpy(), d["oof_prob"].to_numpy()
    for k in (0.05, 0.10, 0.20, 0.30):
        n = int(np.ceil(len(y) * k)); top = np.argsort(-s)[:n]
        rows.append({"product": p, "검사비율": f"{int(k*100)}%", "검사 수": n,
                    "포착 불량": int(y[top].sum()), "전체 불량": int(y.sum()),
                    "포착률": round(y[top].sum() / y.sum(), 3),
                    "무작위 대비 배수": round((y[top].sum() / y.sum()) / k, 2)})
pd.DataFrame(rows)
```

- '무작위 대비 배수' = 포착률 ÷ 검사비율. 예: 20% 검사로 80% 포착 → 4배 효율.
- 보고서 4장 '현장 활용'의 핵심 표. '전수검사 대신 상위 20% 만 검사해도 불량률의 X% 확보' 식으로 쓴다.

[수정 포인트]

- 검사 시나리오는 (0.05, 0.10, 0.20, 0.30) 튜플을 바꾼다.

[셀 29] 4-3. 위험등급 3단계

목적 예측확률 분위수로 정상/주의/위험 등급을 나누고 등급별 실제 불량률을 확인한다.

```
# 위험등급 제안: 확률 분위 기반 3단계
q_hi, q_mid = np.quantile(oof["oof_prob"], [0.90, 0.70])
oof["risk"] = pd.cut(oof["oof_prob"], [-np.inf, q_mid, q_hi, np.inf], labels=["정상", "주의", "위험"])
display(oof.groupby("risk", observed=True).agg(n=(TARGET, "size"), 불량=(TARGET, "sum"),
                                              불량률=(TARGET, "mean")).round(3))
print(f"경계값 - 주의 ≥ {q_mid:.3f}, 위험 ≥ {q_hi:.3f}")
```

- 현재 기준: 상위 10% = 위험, 상위 10~30% = 주의, 나머지 = 정상.
- 등급이 올라갈수록 실제 불량률이 뚜렷하게 커져야 등급 체계가 의미 있다.

[출력 확인]

- 출력된 경계값(확률)을 현장 규칙으로 제시: '확률 ≥ x 이면 즉시 검사'.

[수정 포인트]

- 등급 기준은 `np.quantile(..., [0.90, 0.70])` 의 두 숫자로 조정.

5장. 제출용 예측결과 저장

[셀 30] 5. 테스트 예측결과 저장

목적 제출물 '테스트데이터 예측결과 파일'을 만든다.

```
pred = test[[INDEX_COL, "product", "shot_id", "cavity", TARGET]].copy()
pred["fail_prob"] = p_test
pred["pred"] = (p_test >= THR).astype(int)
pred["inspect_rank"] = pred["fail_prob"].rank(ascending=False, method="first").astype(int)
pred = pred.sort_values("inspect_rank")
pred.to_csv(OUT_DIR / "test_predictions.csv", index=False, encoding="utf-8-sig")
pred.head(15)
```

- 컬럼: 원본 순번, 제품, 사출번호, 캐비티, 실제 정답, fail_prob(불량확률), pred(판정 0/1), inspect_rank(검사 우선순위 1=가장 먼저).
- encoding='utf-8-sig' 는 엑셀에서 한글이 깨지지 않게 하기 위함.

[출력 확인]

- outputs/test_predictions.csv 가 갱신된다 (run_all.py 결과와 같아야 정상).

부록 1. 용어 정리

용어	뜻	이 프로젝트에서
사출(shot) / 캐비티	금형에 수지를 한 번 주입하는 것 / 금형 안의 제품 자리	1회 사출에 제품 2개, 불량 86% 가 캐비티 0
그룹 CV	묶음(그룹)이 train/valid 에 쪼개지지 않게 나누는 교차검증	그룹 = 제품_사출번호
OOF 예측	해당 행이 학습에 안 쓰인 fold 에서 나온 예측	오류분석(3장)의 입력
PR-AUC	정밀도-재현율 곡선 아래 면적. 무작위 = 불량률	주지표. 무작위 0.018
F1	정밀도와 재현율의 조화평균 (임계값 필요)	학습 fold 내부에서 임계값 선정
Recall@k%	확률 상위 k% 만 검사할 때 잡는 불량 비율	검사 우선순위 지표
FN / FP	미탐지(불량→양품) / 오경보(양품→불량)	3장 오류분석 대상
SHAP	각 변수가 예측을 얼마나 올리고 내렸는지 분해	영향변수-상호작용 근거
확률 보정	예측확률이 실제 비율과 맞도록 조정	창의성 항목 재료

부록 2. 자주 나는 오류

증상	원인 · 해결
ModuleNotFoundError: No module named 'src'	노트북을 injection_molding 폴더가 아닌 곳에서 열었다. 폴더에서 jupyter 실행.
numpy / pandas DLL load failed	miniconda 기본 환경 문제. anaconda3 커널로 바꾼다.
그래프 한글이 □ 로 깨짐	font.family 가 설치되지 않은 글꼴. Windows = 'Malgun Gothic'.
SHAP 셸이 너무 오래 걸림	X_dev.sample(500, random_state=0) 로 표본만 설명.
설정을 바꿨는데 2장 결과가 그대로	outputs/cv_folds.csv 를 지우고 셀 0-2 재실행 (또는 python run_all.py).
qcut 에서 에러 / 구간이 적게 나옴	같은 값이 많은 변수. duplicates='drop' 유지, 구간 수 q 를 줄인다.

부록 3. 제출 전 체크리스트

- 새 가상환경에서 pip install -r requirements.txt → python run_all.py 가 에러 없이 끝나는가
- 노트북을 '전체 재시작 후 모두 실행' 해서 출력이 저장된 상태인가
- 출력·README·파일명에 학교명/소속/절대경로(C:\Users\W...)가 없는가 (블라인드 심사)
- outputs/test_predictions.csv 가 zip 에 포함되었는가
- 보고서의 수치가 cv_summary.csv / test_metrics.json 과 일치하는가
- 이 해설지 PDF 는 제출 zip 에서 제외했는가